PROJECT AND TEAM INFORMATION

Project Title

Smart Campus Navigation System – A Web App for Graphic Era Using Dijkstra's Shortest Path

Team Name: *Team #*	*DSCPP-III-2026-T013*
Team member 1 (Team Lead) *(Last Name, name: student ID: email, picture):*	*Aggarwal, Vanshika – 2510011410* *2510011410@geu.ac.in* 
Team member 2 *(Last Name, name: student ID: email, picture):*	*Setia, Suvidha – 2510370306* *2510370306@geu.ac.in* 
Team member 3 *(Last Name, name: student ID: email, picture):*	*Aggarwal, Parishi – 2510011423* *2510011423@geu.ac.in* 
Team member 4 *(Last Name, name: student ID: email, picture):*	*Taneja, Khushi – 2510379518* *2510379518@geu.ac.in* 

PHASE-II PROJECT PROGRESS DESCRIPTION

Project Abstract

- Main goal: build a Campus Navigation System web app for Graphic Era that guides a person from one building to another.
- The user enters only two inputs: the starting building name and the destination building name. Floors, rooms and halls are not part of the system.
- The system finds the shortest available path between the two buildings using Dijkstra's algorithm.
- The result is shown back to the user as a graph of the shortest route, with the buildings on the path in order.
- The campus is modelled as a weighted graph: buildings are nodes, walkable connections are edges, and distances in metres are the edge weights.
- Target users are freshers, visitors, parents and new faculty who do not know the campus and should not have to keep asking for directions.
- Phase I planned a C++ console program. Phase II extends it into a web app, with the C++ backend now complete and the frontend in progress.

Updated Project Approach and Architecture

Approach

- Ground-work: the team walked the campus and prepared a physical map of the buildings and the paths between them.
- Coordinates of 19 locations were collected using Google Maps and written into the backend files.
- The physical map was converted into a virtual map: a weighted graph of 19 nodes and 27 edges (distances in metres).
- The project has two folders: backend (C++) and frontend (HTML, CSS, JavaScript).

Architecture: what happens after the user enters the start and destination

1. index.html shows two input fields (start building, destination building) and a Find Route button. style.css styles this page.
2. script.js reads the two building names and sends them to the backend as an HTTP request in JSON format.
3. main.cpp receives the two names and checks both with hasLocation(). If a name is not recognised, an error message is sent back.
4. main.cpp calls dijkstra(start, end) on the Graph object. Graph.h declares the Graph class (a map of locations and an adjacency list). graph.cpp implements addLocation, addEdge and dijkstra, using a min-heap priority queue and a prev map.
5. Location.h and location.cpp define each building as a Location object (name, latitude, longitude, description), which is stored inside the Graph.
6. Dijkstra returns the ordered list of building names on the shortest path, and main.cpp sends this list back as JSON.
7. script.js receives the list and passes it to map.js. map.js draws the campus graph and highlights the shortest path, which is displayed on index.html.

Libraries, protocols and design

- C++ STL: string, vector, map, queue, limits, algorithm, iostream. Frontend: HTML, CSS, JavaScript.
- Communication: HTTP request/response with JSON data (client-server).
- Design: modular and object-oriented, with Location, Graph and the main driver as separate files.

Tasks Completed

Task Completed	Team Member
Ground-work and physical map of the campus, with building names and paths marked	All members
Collected Google Maps coordinates for 19 locations	All members
Location.h and location.cpp: Location class (name, latitude, longitude, description, getters)	Parishi Aggarwal
Graph.h: Graph class declaration (location map, adjacency list, function prototypes)	Suvidha Setia
graph.cpp: addLocation, addEdge, printGraph and Dijkstra's algorithm	Vanshika Aggarwal
main.cpp: campus data (19 nodes, 27 edges), user input, validation, route output	Khushi Taneja
Complete backend is done: all files compile and run together	All members

Challenges/Roadblocks

General challenges faced while building the system

- Walking the campus and deciding which buildings are really connected and which paths students actually use.
- Getting accurate coordinates from Google Maps, since the pin position can be a few metres off the real building entrance.
- Estimating distances between buildings, because straight-line distance is not the same as walking distance.
- Understanding and implementing Dijkstra's algorithm with a priority queue, and reconstructing the path with a prev map.
- Dividing work across four members and joining the files into one working backend.

Current roadblocks

- Path is not really allocated: the shortest path between coordinates sometimes follows a connection that does not exist on the ground.
- Wrong paths: for some pairs the route is wrong, e.g. the system makes the user go through the Mechanical block first.
- Not the most optimised path: some routes are not the best one a student would take, because the edge weights come from approximate distances.

Plan to solve

- Re-check the physical map and remove edges that are not real walkable paths.
- Re-measure distances along the real path and add junction points where paths turn.
- Re-run all test cases after every correction.

Tasks Pending

Task Pending	Team Member (to complete the task)
PPT (final presentation)	Suvidha Setia
Documentation	Khushi Taneja
Frontend: index.html (input page)	Vanshika Aggarwal
Frontend: style.css (styling)	Suvidha Setia
Frontend: script.js (input handling, backend connection)	Parishi Aggarwal
Frontend: map.js (drawing the route graph)	Khushi Taneja
Fix wrong paths and make the optimised flow path	Vanshika Aggarwal

Project Outcome/Deliverables

- A working Campus Navigation Web App for Graphic Era.
- Input: only the starting building name and the destination building name. No floors, room names or any other detail.
- Output: the shortest path between the two buildings, shown as a graph of the route.
- A C++ backend with the Location class, the Graph class and Dijkstra's algorithm.
- A campus graph of 19 buildings and 27 connections, built from the physical map and Google Maps coordinates.
- Frontend files: index.html, style.css, script.js, map.js.
- Test cases with pass and fail results.
- Final PPT, documentation and project report.

Progress Overview

Completed

- Backend is done: Location class, Graph class, Dijkstra's algorithm and the main driver are complete.
- System architecture is done.
- Workflow is done: input $\rightarrow$ validation $\rightarrow$ Dijkstra $\rightarrow$ route output.
- Report is done: Phase I proposal and this Phase II progress report.
- Physical map and Google Maps coordinates are done, and the campus graph (19 nodes, 27 edges) is built.

Remaining tasks

- Frontend: index.html, style.css, script.js, map.js, split between the four members.
- PPT: Suvidha.
- Documentation: Khushi.
- Optimised flow path: fix the wrong and non-optimal routes found during testing.

Schedule status

- On track: backend, architecture and workflow.
- Behind schedule: optimised path accuracy and the frontend.
- Next: correct the graph data, build the frontend, then prepare the PPT and documentation.

Codebase Information

- Repository: [https://github.com/suvidhasetia14-creator/DSCPP-III-2026-T013]
- Branch: main
- The codebase has 2 folders: backend and frontend.

backend folder (C++)

- Location.h: declares the Location class (name, latitude, longitude, description and getters).
- location.cpp: implements the constructor and getters of the Location class.
- Graph.h: declares the Graph class (map of locations, adjacency list and function prototypes).
- graph.cpp: implements addLocation, hasLocation, addEdge, printGraph and Dijkstra's algorithm.
- main.cpp: holds the campus data (19 locations, 27 edges), takes the start and destination, validates them, runs Dijkstra and returns the route.

frontend folder (HTML, CSS, JavaScript)

- index.html: page with the start and destination inputs and the route display area.
- style.css: styling and layout of the page.
- script.js: reads the inputs, sends them to the backend and receives the route.
- map.js: draws the campus graph and highlights the shortest path.

Important commits

- Location class added; Graph class and edges added; Dijkstra's algorithm added; campus data and user input added.

Testing and Validation Status

ID	Test (Start → Destination)	Output Path	Status	Notes
T1	Quick Bite Cafe → CS/IT Block	Quick Bite Cafe → Old MCA Block → MCA Block → Param Computer Lab → Santoshanand Library → GEU Main Block → Graphic Era Main Ground → CS/IT Block	Pass	Most optimised path (497 m).
T2	Gate 2 → CS/IT Block	Gate 2 → Graphic Era Main Ground → CS/IT Block	Pass	Direct route (458 m).
T3	Gate 2 → Santoshanand Library	Gate 2 → Bike Parking → Dept. of Mechanical Engg → Dept. of Management Studies → Santoshanand Library	Pass	Shorter than the ground-side route (305 m vs 472 m).
T4	Bike Parking → Param Computer Lab	Bike Parking → Dept. of Mechanical Engg → Dept. of Management Studies → Santoshanand Library → Param Computer Lab	Pass	Correct shortest path (260 m).
T5	MCA Block → Basketball Court	MCA Block → Param Computer Lab → Santoshanand Library → Digital Lab → Basketball Court	Pass	Correct (190 m).
T6	Old MCA Block → Dept. of Electrical Engg	Old MCA Block → MCA Block → Param Computer Lab → Dept. of Electrical Engg	Pass	Correct (185 m).
T7	Dept. of Mechanical Engg → Petroleum Engg Block	Dept. of Mechanical Engg → Dept. of Management Studies → Dept. of Hotel Management → Petroleum Engg Block	Pass	Correct (132 m).
T8	CS/IT Block → Digital Lab	CS/IT Block → Graphic Era Main Ground → GEU Main Block → Santoshanand Library → Digital Lab	Pass	Correct (297 m).
T9	Gate 2 → GEU Main Block	Gate 2 → Bike Parking → Dept. of Mechanical Engg → Dept. of Management Studies → Santoshanand Library → GEU Main Block	Fail	Forces the user through the Mechanical block first, which is wrong.
T10	Gate 2 → Civil Block	Gate 2 → Bike Parking → Dept. of Mechanical Engg → Dept. of Management Studies → Civil Block	Fail	Detours through Management Studies using a straight-line edge (226 m) that is not a real path.
T11	Badminton Court → Dept. of Hotel Management	Badminton Court → Dept. of Management Studies → Dept. of Hotel Management	Fail	Uses a straight-line edge (244 m) that is not a real walkable path.
T12	Gate 2 → Gate 2 (same place)	Gate 2	Pass	Edge case: returns only the start.
T13	Gate 2 → Library (invalid name)	"One or both locations were not recognized"	Pass	Validation works. Names must match exactly (spelling and case).

Deliverables Progress

- C++ backend (Location, Graph, main): Completed
- Campus graph, 19 nodes and 27 edges: Completed
- Dijkstra's shortest path: Completed (accuracy fixes in progress)
- System architecture and workflow: Completed
- Project report: Completed
- Testing with pass/fail cases: In progress
- Optimised flow path: In progress
- Web frontend (index.html, style.css, script.js, map.js): Pending
- Route graph display: Pending
- PPT (Suvidha): Pending
- Documentation (Khushi): Pending